

# Contents

|                                         | Page No     |
|-----------------------------------------|-------------|
| List of Figures                         | i           |
| List of Tables                          | ii          |
| Abbreviations                           | iii         |
| Abstract                                | iv          |
| <br>                                    |             |
| <b>1. Introduction</b>                  | <b>1-2</b>  |
| 1.1 Introduction of the Project         | 1           |
| 1.2 Existing System                     | 1           |
| 1.3 Problems in Existing System         | 1           |
| 1.4 Proposed System                     | 2           |
| 1.5 Advantages of Proposed System       | 2           |
| <br>                                    |             |
| <b>2. Requirements Analysis</b>         | <b>3-5</b>  |
| 2.1 Functional Requirements             | 4           |
| 2.2 Non-functional Requirements         | 4-5         |
| 2.3 Computational Resource Requirements | 5           |
| 2.3.1 Hardware Requirements             | 5           |
| 2.3.2 Software Requirements             | 5           |
| <br>                                    |             |
| <b>3. Design</b>                        | <b>6-10</b> |
| 3.1 Architecture                        | 7-8         |
| 3.1.1 Software Architecture             | 7           |
| 3.1.2 Technical Architecture            | 8           |
| 3.1.3 Block Diagram.                    | 8-10        |

|           |                                    |              |
|-----------|------------------------------------|--------------|
| <b>4.</b> | <b>Implementation</b>              | <b>11-20</b> |
|           | 4.1 Technologies                   | 12-14        |
|           | 4.1.1 Embedded C                   | 12           |
|           | 4.1.2 Features of Embedded C       | 12-13        |
|           | 4.1.3 Python                       | 13           |
|           | 4.1.4 Features of Python           | 13-14        |
|           | 4.2 Pseudocode                     | 14-20        |
| <b>5.</b> | <b>Testing</b>                     | <b>21-25</b> |
|           | 5.1 Overview                       | 22           |
|           | 5.2 Dimensions of Testing          | 22           |
|           | 5.3 Stages of Testing              | 23-24        |
|           | 5.3.1 Unit Testing                 | 23           |
|           | 5.3.2 Integration Testing          | 23           |
|           | 5.3.3 System Testing               | 23           |
|           | 5.3.4 Acceptance Testing           | 24           |
|           | 5.4 Types of Testing               | 24           |
|           | 5.4.1 Black box Testing            | 24           |
|           | 5.4.2 White box Testing            | 24           |
|           | 5.5 Test Cases                     | 25           |
| <b>6.</b> | <b>Screenshots</b>                 | <b>26-36</b> |
| <b>7.</b> | <b>Conclusion and Future Scope</b> | <b>37-38</b> |
|           | 7.1 Conclusion                     | 37           |
|           | 7.2 Future Scope                   | 37           |
| <b>8.</b> | <b>References</b>                  | <b>39-40</b> |

## List of Figures

| <b>Fig. No</b> | <b>Figure Name</b>                                                                    | <b>Page No</b> |
|----------------|---------------------------------------------------------------------------------------|----------------|
| 3.1            | Software Architecture                                                                 | 7              |
| 3.2            | Technical Architecture                                                                | 8              |
| 3.3            | Block Diagram                                                                         | 9              |
| 3.4            | Screenshots                                                                           | <b>26 - 36</b> |
| 6.1            | Hardware Prototype of IOT Based Gas Leakage Detection System                          | 27             |
| 6.2            | Instead of LPG gas, marker refill is given as input to gas sensor for ensuring safety | 28             |
| 6.3            | Flame is given as input to flame sensor                                               | 29             |
| 6.4            | Exhaustible Fan turned off when gas leakage is not detected                           | 30             |
| 6.5            | Exhaustible Fan turned on when gas leakage is detected                                | 31             |
| 6.6            | LCD Display when no gas leak and no fire is detected                                  | 32             |
| 6.7            | LCD Display when gas leak is not detected but fire is detected                        | 33             |
| 6.8            | LCD Display when gas leak is detected but fire is not detected                        | 34             |
| 6.9            | LCD Display when gas leak is detected and fire is detected                            | 35             |
| 6.10           | IOT Web Page                                                                          | 36             |

## List of Tables

| Table. No | Table Name | Page. No |
|-----------|------------|----------|
| 5.1       | Test cases | 25       |

## **Abbreviations**

LCD: Liquid Crystal Display

LED: Light Emitting Diode

Regulator IC: Regulator Integrated Circuit

RS: Register Select

SPDT: Single Pole Double Throw

MCU – Microcontroller Unit

Wi-Fi – Wireless Fidelity (wireless communication standard)

IOT – Internet of Things

## **Abstract**

The IoT-based Gas Leakage Detection System is a device designed to detect gas leaks and fire hazards promptly. It uses gas and fire sensors connected to a microcontroller (ESP8266) to monitor the environment continuously. Upon detecting a gas leak or fire, the system triggers a buzzer to alert nearby individuals, activates a fan to disperse the leaked gas safely, and switches on water sprinklers to control fire outbreaks. Alerts are also displayed on an LCD screen and sent in real time via Wi-Fi to a web interface for remote monitoring. This integrated approach ensures early detection and rapid response, enhancing safety and preventing potential accidents.

## 1. INTRODUCTION

## 1. Introduction

### 1.1 Introduction of the Project

Liquefied Petroleum Gas (LPG) is a widely used non-renewable energy source derived from petroleum and natural gas. It is highly flammable and must be stored away from ignition sources in well-ventilated areas to prevent accidental accumulation and hazards. Since LPG vapours are heavier than air, they tend to settle in low-lying areas, making leakage detection critical for safety. In its natural state, LPG is odorless however, a distinctive odorant is added to help detect leaks through smell.

This project aims to design and implement an LPG gas leakage detection system using IOT suitable for homes. The system uses an MQ-5 gas sensor to monitor the presence and concentration of LPG in the air. Real-time alerts are displayed via an LCD screen, ensuring timely notification. By integrating this system, the goal is to prevent accidents, minimize property damage, and enhance overall safety in environments where LPG is commonly used.

### 1.2 Existing System

Existing LPG gas leakage detection systems are generally simple and use basic gas sensors like the MQ-2 to detect the presence of gas. These systems are designed to alert users locally when the gas concentration exceeds a certain safety threshold. The alert is typically given through a buzzer sound or an LED indicator.

### 1.3 Problems in Existing System

- There is no automatic fan or sprinkler system to reduce gas concentration or extinguish flames.
- Alerts are limited to basic buzzer sounds or LED lights. There is no support for real-time monitoring through a web interface.
- The system cannot detect fire.



## 1.4 Proposed System

The proposed LPG gas leakage detection system is developed to provide enhanced safety, especially in household environments. It uses a gas sensor to continuously monitor for gas leaks. When a gas leak or fire is detected, the system immediately activates a buzzer and displays the alert on an LCD screen to notify the user.

To prevent gas accumulation, an exhaust fan is automatically turned on, helping to disperse the gas. In the event of fire detection, the system triggers a water sprinkler to control flames and minimize damage. Additionally, the system features a Wi-Fi-enabled web page (accessible via 192.168.4.1) that displays real-time gas and fire status, allowing users to monitor safety conditions from a connected device.

This integrated approach improves response time, reduces risk, and ensures a higher level of protection against gas-related hazards.

## 1.5 Advantages of Proposed System

- The system activates a buzzer and shows alerts on the LCD screen. This ensures the user is immediately notified of any gas leakage.
- A fan is automatically turned on when gas is detected. This helps disperse the leaked gas and lowers the risk of ignition.
- Upon detecting fire, the sprinkler system is triggered. It releases water to control flames and prevent further damage.
- A real-time web interface (192.168.4.1) is provided for monitoring. Users can view gas and fire status from any connected device.
- The system responds to both gas leaks and fire hazards. This dual-action approach greatly enhances overall safety.

## 2. REQUIREMENT ANALYSIS

## 2. Requirement Analysis

### 2.1 Functional Requirements

An IoT-based gas leakage detection system uses Arduino module that identifies gas leaks early and alert users automatically. By leveraging sensors and IoT technology, the system ensures safety in households.

An IOT Based Gas Leakage Detection System consists of User Module only.

#### 1.User Module:

Users start by powering on the system, which activates the sensor, LCD, buzzer, fan, and sprinkler. Alerts are shown on the LCD to the user, and a buzzer provides sound warnings during gas leaks or fire.

- Initial Setup: Connect system to power supply.
- Users can view alerts through LCD and buzzer sounds.
- Regularly test sensors, buzzer, fan, LCD, and other components for reliable operation.

### 2.2 Non-Functional Requirements

- Usability: The system should be simple to operate with clear LCD messages and buzzer alerts. Users must understand warnings instantly without needing technical knowledge.
- Maintainability: Hardware components and software code should be easy to update or replace. This ensures long-term use and simplifies troubleshooting and upgrades.
- Reliability: The system must consistently detect gas and fire with minimal false alarms. It should work accurately under all environmental conditions.

- **Performance:** Alerts and automated actions must be triggered within seconds of leak detection .Quick response helps minimize risk and prevent accidents effectively.
- **Security:** Communication and user data should be securely encrypted.
- **Scalability:** The system should support adding more sensors or modules as needed. It must be flexible enough to adapt to larger areas or new features.
- **Portability:** The hardware design should be compact and lightweight. It should be easy to relocate and install in different environments.
- **Availability:** The system should operate continuously, 24/7, without failures or downtime. This ensures safety in all times.

## 2.3 Computational Resource Requirements

### 2.3.1 Hardware Requirements

- Processor : ATmega328P
- Microcontroller : Arduino Uno R3

### 2.3.2 Software Requirements

- Programming Language : Embedded C, Python
- IDE : Arduino

### 3. DESIGN

## 3. Design

### 3.1 Architecture

Project architecture represents number of components we are using as a part of our project and the flow of request processing i.e. what components in processing the request and in which order. An architecture description is a formal description and representation of a system organized in a way that supports reasoning about the structure of the system. Architecture is of two types. They are:

- (1) Software Architecture
- (2) Technical Architecture

#### 3.1.1 Software Architecture

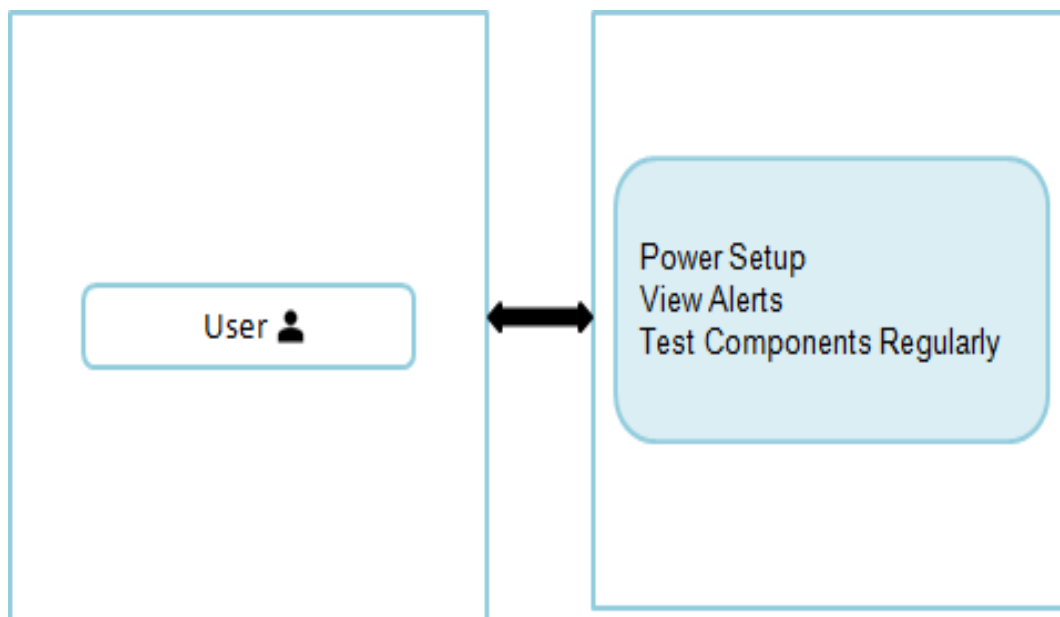


Fig 3.1 Software Architecture

### 3.1.2 Technical Architecture

The Technical Architecture of the project is as follows:

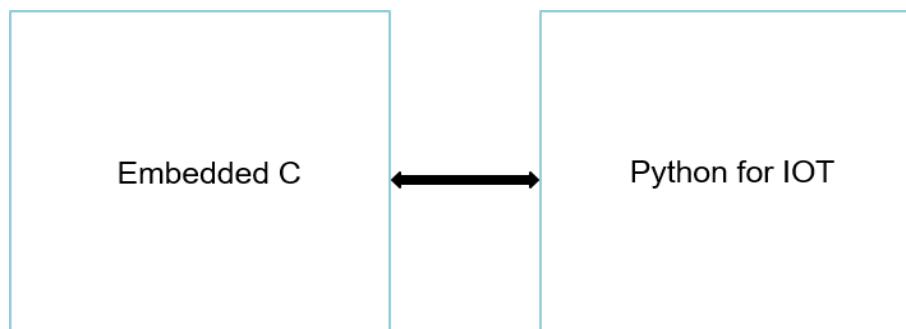


Fig 3.2 Technical Architecture

### 3.1.3 Block Diagram

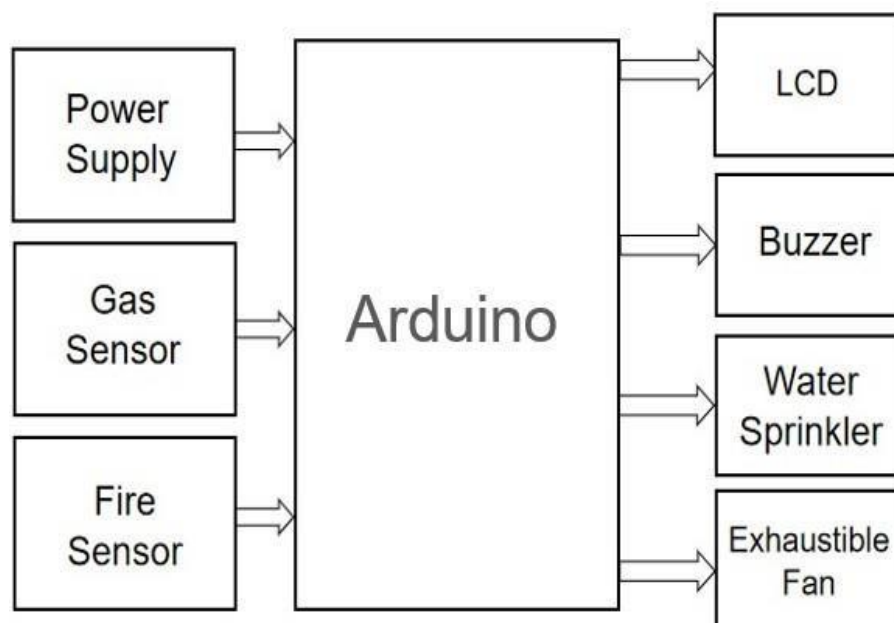


Fig 3.3 Block Diagram

The above block diagram represents the architecture of the LPG Gas Leakage and Fire Detection System using an Arduino microcontroller.

#### Working of Power Supply

The power supply converts 230V AC to 5V DC needed for components like Arduino and sensors. A transformer reduces voltage, and a rectifier changes AC to DC. A capacitor smoothes the output, and the 7805 regulator ensures stable 5V.

#### Working of Arduino

Arduino acts as the control unit of the LPG gas leakage detection system. It continuously monitors sensor inputs. When gas levels exceed the safe limit, Arduino executes pre-written instructions to initiate safety measures. It triggers a buzzer to produce sound alerts and activates a fan to reduce gas concentration. If a flame is also detected, it turns on a water sprinkler for safety.

#### Working of Gas Sensor

The MQ-5 sensor detects gases like LPG using a chemical layer that changes resistance. When gas is present, resistance drops, creating a voltage signal. Arduino reads this signal and responds accordingly.

#### Working of LCD

The 16x2 LCD module is used for displaying real-time alerts and messages. It has two lines and 16 characters per line. The LCD is connected to the Arduino through pins like RS (register select), R/W (read/write), and EN (enable), along with data lines. Characters are stored in its DDRAM and shown using a 5x7 dot matrix. The LCD displays messages such as “Gas Status: YES/NO” and “Fire Status: YES/NO” to inform users immediately. Its clear visual output helps in quick decision-making during emergencies.



### Working of Buzzer

The buzzer provides an audible alert in case of gas leakage or fire. It uses a piezoelectric element that vibrates and produces sound when electricity passes through it. The Arduino sends a HIGH signal to the buzzer pin once a gas leak or fire is detected. This signal turns on the buzzer, alerting nearby individuals. This helps in catching attention quickly, even if the person is not near the display.

### Working of Flame Sensor

The flame sensor detects fire by sensing infrared (IR) light emitted from flames, especially in the 760 nm to 1100 nm range. It includes an IR receiver and an LM393 comparator to evaluate light intensity. When a flame is within about 80 cm range, it sends a HIGH signal to the Arduino. This signal informs the system about a possible fire. In response, the Arduino can activate devices like the buzzer and a water sprinkler to prevent fire spread.

### Working of Wi-Fi Module

The ESP8266 Wi-Fi module connects the system to the internet. It sends real-time alerts to a web page when gas or fire is detected. This helps users monitor the system remotely.

## 4. IMPLEMENTATION

## 4. Implementation

### 4.1 Technologies

This system is developed using Embedded C language and Python.

#### 4.1.1 Embedded C

Embedded C is a set of language extensions for the C programming language, specifically designed for developing software for embedded systems. Embedded systems are computing units that perform dedicated functions within larger systems commonly found in devices such as washing machines, medical instruments, traffic lights, and micro-controller based hardware projects.

Embedded C bridges the gap between low-level hardware interaction and high-level logic control, making it ideal for writing programs that run on micro-controllers and microprocessors.

Embedded C is used to program the Arduino, enabling it to read sensor data from gas and fire detectors. It processes this data in real-time and controls outputs like the buzzer, fan, LCD, and sprinkler. Its low-level hardware access ensures fast and reliable system response during emergencies. Overall, it provides the core logic that makes the gas leakage detection system function effectively.

#### 4.1.2 Features of Embedded C

- **Hardware-Level Access:** Allows direct interaction with micro-controller registers, memory, and I/O pins.
- **Efficiency:** Optimized for performance in low-resource environments with limited memory and processing power.
- **Portability:** Code can be adapted to various micro-controllers with minimal changes.

- 
- **Real-Time Operation:** Suitable for time-critical applications that require immediate responses.
  - **Modularity:** Supports structured programming with functions and modules for better maintainability.
  - **Predictability:** Ensures consistent execution, which is vital for safety-critical systems.
  - **Integration with Assembly:** Can integrate low-level assembly code for fine-tuned hardware control.
  - **Wide Hardware Support:** Compatible with a large range of embedded platforms like AVR, ARM, PIC, and 8051.

#### **4.1.3 Python**

Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability. It is widely used in various domains such as web development, data science, machine learning, automation, embedded systems, and Internet of Things (IOT).

Python is platform-independent and supports multiple programming paradigms including object-oriented, procedural, and functional programming. Its easy-to-learn syntax makes it a popular choice for both beginners and professionals.

#### **4.1.4 Features of Python**

- **Simple and Readable Syntax:** Python code is easy to read and write, making it ideal for rapid development and maintenance.
- **Interpreted Language:** Python does not require compilation, which makes development and testing faster.
- **Cross-Platform Compatibility:** Python programs can run on various operating systems like Windows, Linux, and macOS without modification.

- 
- Extensive Standard Library: Comes with a large set of built-in modules for handling strings, files, networking, math, and more.
  - Dynamic Typing: Variable types are determined at runtime, allowing more flexibility in coding.
  - Object-Oriented and Modular: Supports classes and objects for structured programming and code reuse.
  - Large Community and Libraries: A vast ecosystem of third-party libraries for data science (NumPy, pandas), AI (TensorFlow, PyTorch), web (Flask, Django), IOT, and more.
  - Integration and Extensibility: Can be integrated with other languages like C, C++, and Java for performance-critical applications.

## 4.2 Pseudocode

### Embedded c code

```
#include <SoftwareSerial.h>
#include <LiquidCrystal.h>

// LCD pin configuration
const int rs = 13, en = 12, d4 = 11, d5 = 10, d6 = 9, d7 = 8;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

// Sensor and actuator pin definitions
const int gasSensorPin = A2;    // Gas sensor input
const int flameSensorPin = A3;  // Flame sensor input
const int motorPin = 6;         // Exhaust fan (motor)
const int buzzerPin = 5;        // Buzzer
const int sprinklerPin = 4;     // Sprinkler system
```

```
// SoftwareSerial pins to communicate with ESP01
int serialRx = A0;
int serialTx = A1;
SoftwareSerial portOne(serialRx, serialTx); // RX, TX

// Variables for communication
String inMsg = "", Msg = "";
int responseTime = 10;

void setup() {
    Serial.begin(115200);    // Serial monitor
    portOne.begin(115200);   // ESP01 communication

    // Initialize LCD
    lcd.begin(16, 2);
    lcd.print("LPG GAS LEAKAGE");
    lcd.setCursor(0, 1);
    lcd.print("USING IOT");
    delay(2000);
    lcd.clear();

    // Set pin modes
    pinMode(gasSensorPin, INPUT);
    pinMode(flameSensorPin, INPUT);
    pinMode(motorPin, OUTPUT);
    pinMode(buzzerPin, OUTPUT);
    pinMode(sprinklerPin, OUTPUT);

    // Initialize outputs
    digitalWrite(motorPin, LOW);
    digitalWrite(buzzerPin, LOW);
```

---

```
digitalWrite(sprinklerPin, LOW);
// ESP module setup commands
sendToWifi("AT+CIPAP_CUR?", responseTime); // Get IP
sendToWifi("AT+CIPSTO=600", responseTime); // Set timeout
}

void loop() {
    // Read sensors
    bool gasDetected = (digitalRead(gasSensorPin) == LOW);    // LOW means
gas detected
    bool fireDetected = (digitalRead(flameSensorPin) == LOW); // LOW means
flame detected

    // Display status on LCD
    lcd.setCursor(0, 0);
    lcd.print("GAS STATUS: ");
    lcd.print(gasDetected ? "YES " : "NO ");

    lcd.setCursor(0, 1);
    lcd.print("FIRE STATUS: ");
    lcd.print(fireDetected ? "YES" : "NO ");

    // Actuator control based on detection
    digitalWrite(buzzerPin, gasDetected || fireDetected);
    digitalWrite(motorPin, gasDetected);    // Fan ON if gas is detected
    digitalWrite(sprinklerPin, fireDetected); // Sprinkler ON if fire is detected

    // Send sensor status to ESP
    if (gasDetected)
        sendData("GAS LEAKAGE\r");
    else
        sendData("NO GAS LEAKAGE\r");
```

---

```
    if (fireDetected)
        sendData("FIRE DETECTED\r");
    else
        sendData("NO FIRE DETECTED\r");

    delay(1000); // Wait before next loop
}

// Send AT command to ESP-01
void sendToWifi(String command, const int timeout) {
    Serial.print("Sending to ESP: ");
    Serial.println(command);
    portOne.println(command);
    long int time = millis();
    while ((time + timeout) > millis()) {
        // just wait
    }
}

// Read incoming data from ESP-01 (optional use)
String readFromWifi() {
    char arrayInMsg[100];
    String tempStr;
    int count = 0;
    while (portOne.available() > 0) {
        arrayInMsg[count] = portOne.read();
        if ((arrayInMsg[count] == '\n') || (arrayInMsg[count] == '\r')) break;
        count++;
        delay(50);
    }
}
```



```
    arrayInMsg[count] = '\0';  
    tempStr = String(arrayInMsg);  
    tempStr.trim();  
    return tempStr;  
}  
  
// Send sensor data string to ESP-01  
void sendData(String str) {  
    portOne.print(str);  
    Serial.print("Sent to IoT: ");  
    Serial.println(str);  
}
```

Python code for IOT Web Page

```
from flask import Flask, render_template_string
```

```
app = Flask(__name__)
```

```
gas_status = "Safe" fire_status = "Safe"
```

```
html_template = """
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>IOT LPG SAFETY SYSTEM</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif; text-align: center;
```

```
      margin-top: 80px;
```

```
    }
```

```
    h1 {
```

```
      color: red;
```

```
    }
```

```
    p {
```

```
      font-size: 22px;
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
    <h1>IOT BASED LPG SAFETY SYSTEM</h1>
    <p><strong>GAS Status:</strong> {{ gas_status }}</p>
    <p><strong>FIRE Status:</strong> {{ fire_status }}</p>
</body>
</html>
''''''
```

```
@app.route("/")
def index():
    return render_template_string(html_template,gas_status=gas_status,
fire_status=fire_status)
@app.route("/update/<gas>/<fire>")
def update_status(gas, fire):
    global gas_status, fire_status
    gas_status = gas
    fire_status = fire
    return "Status Updated"

if __name__ == "__main__":
    app.run(host="192.168.4.1", port=80, debug=True)
```

## 5. TESTING

## 5. Testing

### 5.1 Overview

Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

As per the current trend, due to constant change and development in digitization, our lives are improving in all areas. The way we work is also changed. We access our bank online; we do shop online; we order food online and many more. We rely on software's and systems. What if these systems turn out to be defective? We all know that one small bug shows huge impact on business in terms of financial loss and goodwill. To deliver a quality product, we need to have Software Testing in the Software Development Process.

Some of the reasons why software testing becomes very significant and integral part in the field of information technology are as follows.

1. Cost effectiveness
2. Customer Satisfaction
3. Security
4. Product Quality

### 5.2 Dimensions of Testing

There are many different dimensions to consider:

1. Layers of the application (database, APIs, UI)
2. Scale of testing (unit, module, integration, scenario)
3. Type of testing (functional, performance, security, etc.)
4. Methodology (exploratory, scripted manual, automated)

## 5.3 Stages of Testing

### 5.3.1 Unit Testing

Unit testing involves testing individual components of the system to ensure they function correctly in isolation. In this project, the gas sensor, fire sensor, buzzer, motor fan, sprinkler, and LCD display were each tested independently. For example, the gas sensor was tested by bringing a marker refill close to it (as a substitute for LPG gas) and checking whether it produced a low analog value indicating a gas leak. The fire sensor was tested using a lighter flame to ensure it correctly detected fire and changed the digital output. Similarly, the buzzer, motor, and sprinkler were tested by manually triggering them through the code to confirm they activated and deactivated as expected. The LCD was checked for correct initialization and display updates.

### 5.3.2 Integration Testing

Integration testing was carried out to ensure the various hardware components and software modules work together seamlessly. For instance, when a gas leak was simulated using a marker, the gas sensor was expected to trigger the fan and buzzer. Similarly, when fire was detected by the sensor, the sprinkler and buzzer should activate, and the LCD and web interface should update accordingly. This testing confirmed that sensors, actuators, display, and web communication via the ESP8266 module were integrated properly and responded accurately to real-time inputs.

### 5.3.3 System Testing

System testing focused on evaluating the entire LPG gas and fire detection setup as a complete working system. This included powering on the device, ensuring the Wi-Fi Access Point was created successfully, checking the LCD for proper boot messages, and verifying real-time status updates on both LCD and web page. Different scenarios were tested, such as gas leak only, fire only, and both conditions simultaneously. The system behaved as expected in all scenarios, with accurate sensor detection, actuator activation, and synchronization between LCD and web interface. The reliability and timing of alerts were also validated under normal and stressed conditions.

### **5.3.4 Acceptance Testing**

Acceptance testing was conducted to ensure that the system meets the requirements and performs its intended safety functions effectively. This testing involved simulating real-world scenarios in front of potential users or supervisors. For example, when a leak or fire was introduced, the system had to detect it immediately, activate necessary actuators like the buzzer and sprinkler, and alert users through the LCD and the IoT web interface. The system passed acceptance criteria by accurately detecting dangers, responding promptly, and presenting user-friendly and clear status messages. This confirmed that the project is ready for deployment in small-scale environments like kitchens or labs for LPG safety.

## **5.4 Types of Testing**

### **5.4.1 Black box Testing**

Black box testing, on the other hand, focuses on testing the system's external behavior without any knowledge of the internal code. In this project, it involved giving inputs like a marker refill (as a substitute for LPG gas) or a small flame and observing whether the system responded correctly. We verified that the buzzer, fan, sprinkler, LCD, and web page output the right alerts when a leak or fire was detected. This type of testing ensured that the user experience and overall system functionality met expectations, regardless of how the code was written internally.

### **5.4.2 White box testing**

White box testing involves testing the internal logic and structure of the code. It is performed with complete knowledge of the source code, allowing the tester to verify the control flow, conditions, and function outputs. In our LPG gas leakage detection project, white box testing was used to check how the code responds to different gas and fire sensor values. The logic that controls the buzzer, fan, sprinkler, and LCD display was carefully traced and tested for correct functioning. Even the function that generates the web page was reviewed to ensure it outputs the right HTML content based on the sensor readings.

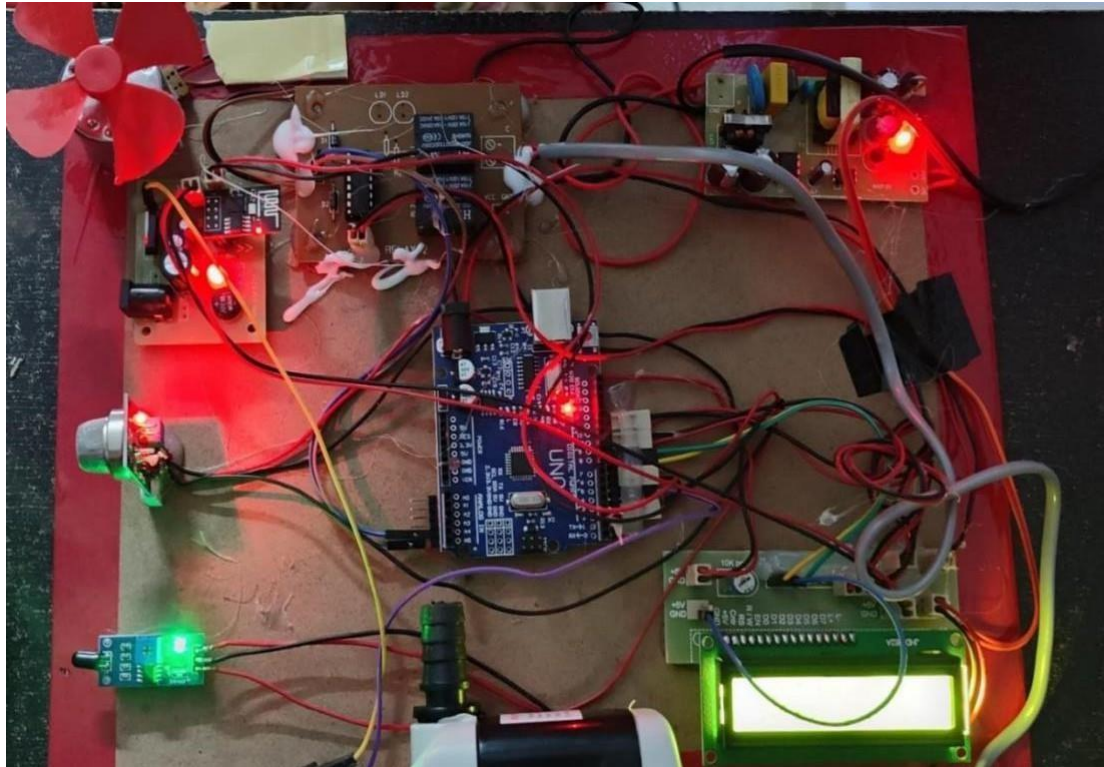
## 5.5 Test Cases

| Test ID | Test Scenario                    | Input                                                               | Expected Output                                                                              | Actual Output                                                                                | Status |
|---------|----------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------|
| 1       | No Gas Leak and No Fire Detected | No input is given to gas and fire sensors                           | LCD- Gas Status: NO<br>Fire Status: NO<br>Buzzer - OFF<br>Fan - OFF<br>Water Sprinkler - OFF | LCD- Gas Status: NO<br>Fire Status: NO<br>Buzzer - OFF<br>Fan - OFF<br>Water Sprinkler - OFF | Pass   |
| 2       | Gas Leak and No Fire Detected    | LPG gas (Marker pen refill)                                         | LCD- Gas Status: YES<br>Fire Status: NO<br>Buzzer - ON<br>Fan - ON<br>Water Sprinkler – OFF  | LCD- Gas Status: YES<br>Fire Status: NO<br>Buzzer - ON<br>Fan - ON<br>Water Sprinkler – OFF  | Pass   |
| 3       | No Gas Leak but Fire Detected    | Flame (Matchstick) near Flame Sensor                                | LCD- Gas Status: NO<br>Fire Status: YES<br>Buzzer - ON<br>Fan - OFF<br>Water Sprinkler – ON  | LCD- Gas Status: NO<br>Fire Status: YES<br>Buzzer - ON<br>Fan - OFF<br>Water Sprinkler – ON  | Pass   |
| 4       | Gas Leak and Fire Detected       | LPG gas (Marker pen refill)<br>Flame (Matchstick) near Flame Sensor | LCD- Gas Status: YES<br>Fire Status: YES<br>Buzzer - ON<br>Fan - ON<br>Water Sprinkler – ON  | LCD- Gas Status: YES<br>Fire Status: YES<br>Buzzer - ON<br>Fan - ON<br>Water Sprinkler – ON  | Pass   |

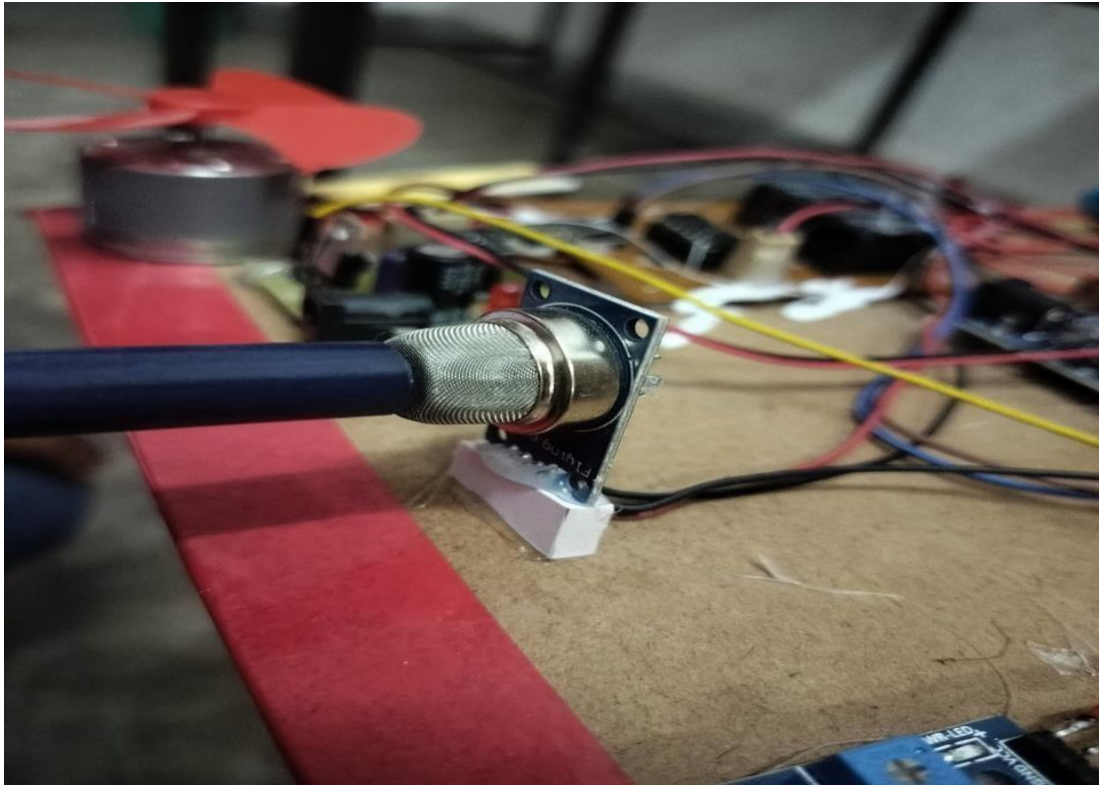
Table 5.1 Test Cases



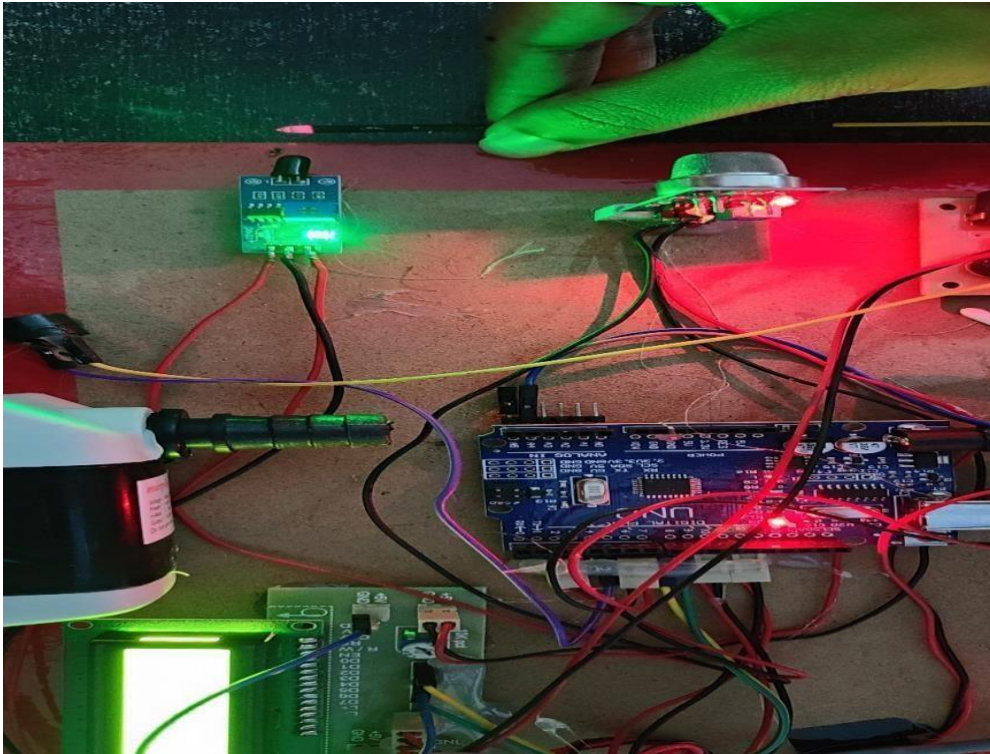
## 6. SCREENSHOTS



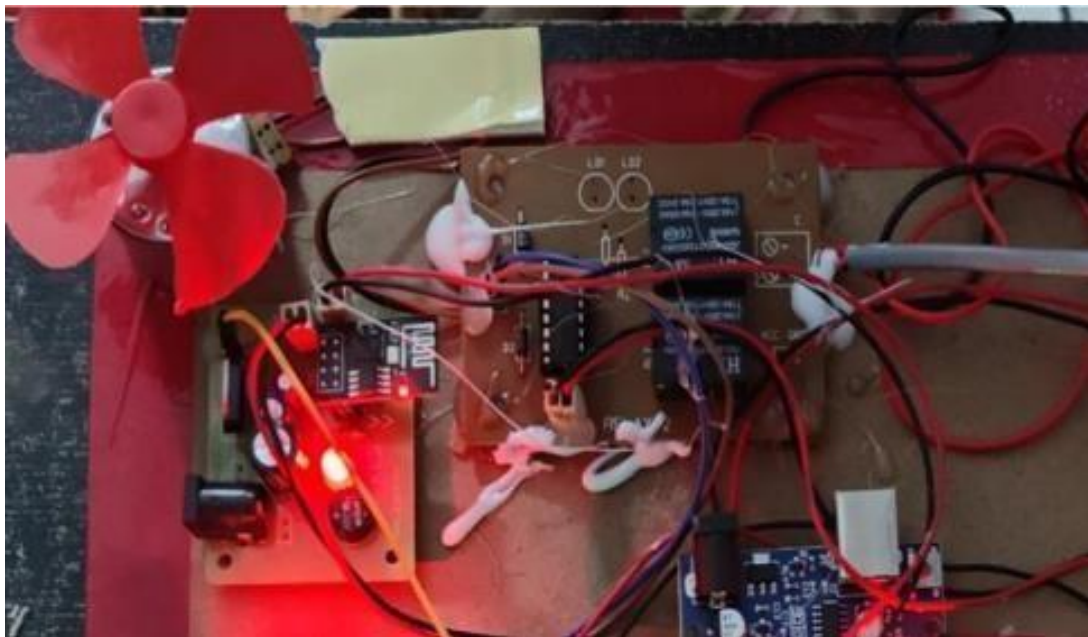
Screenshot 6.1 Hardware Prototype of IOT Based Gas Leakage Detection System



Screenshot 6.2 For practical and safety reasons, marker pen refill is used instead of actual LPG gas during experimentation

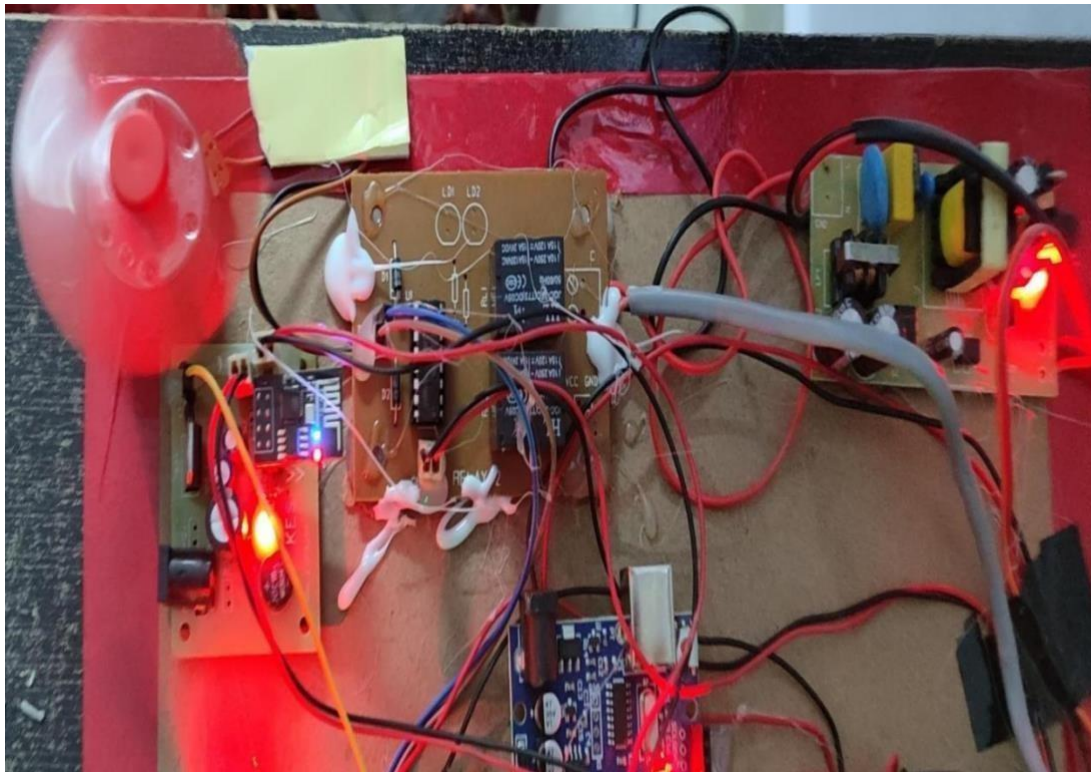


Screenshot 6.3 Flame is given as input to flame sensor

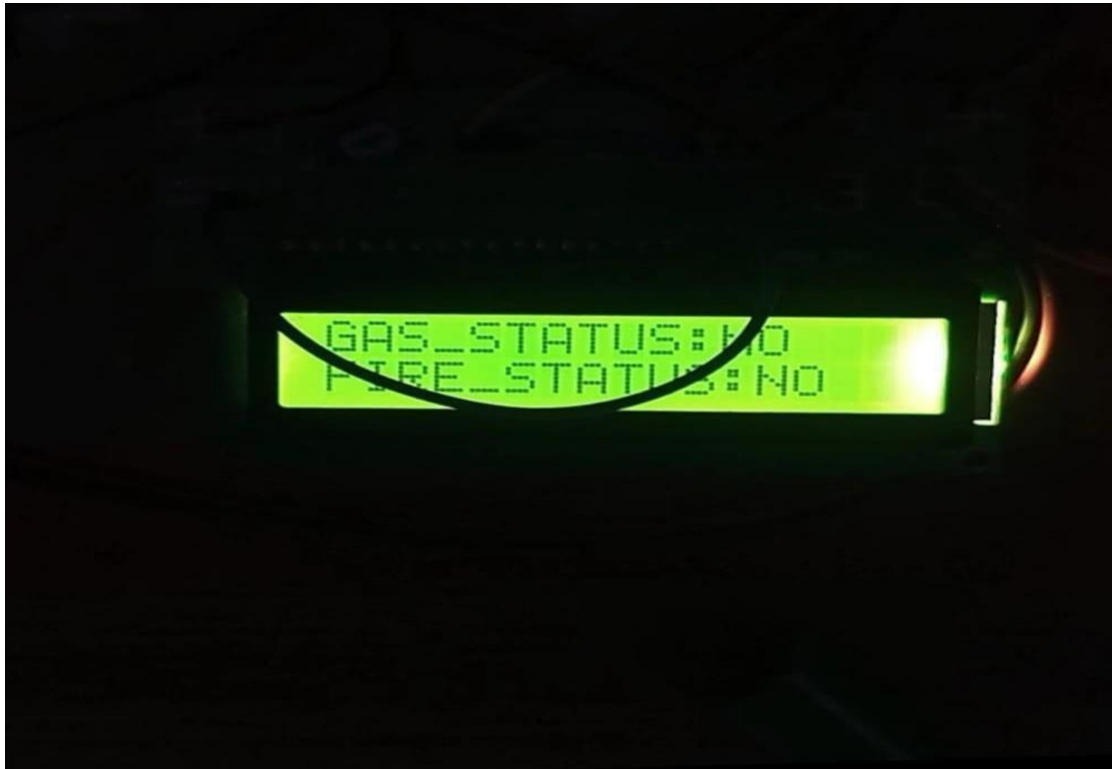


Screenshot 6.4 Exhaustible Fan is turned off when Gas leakage is not detected

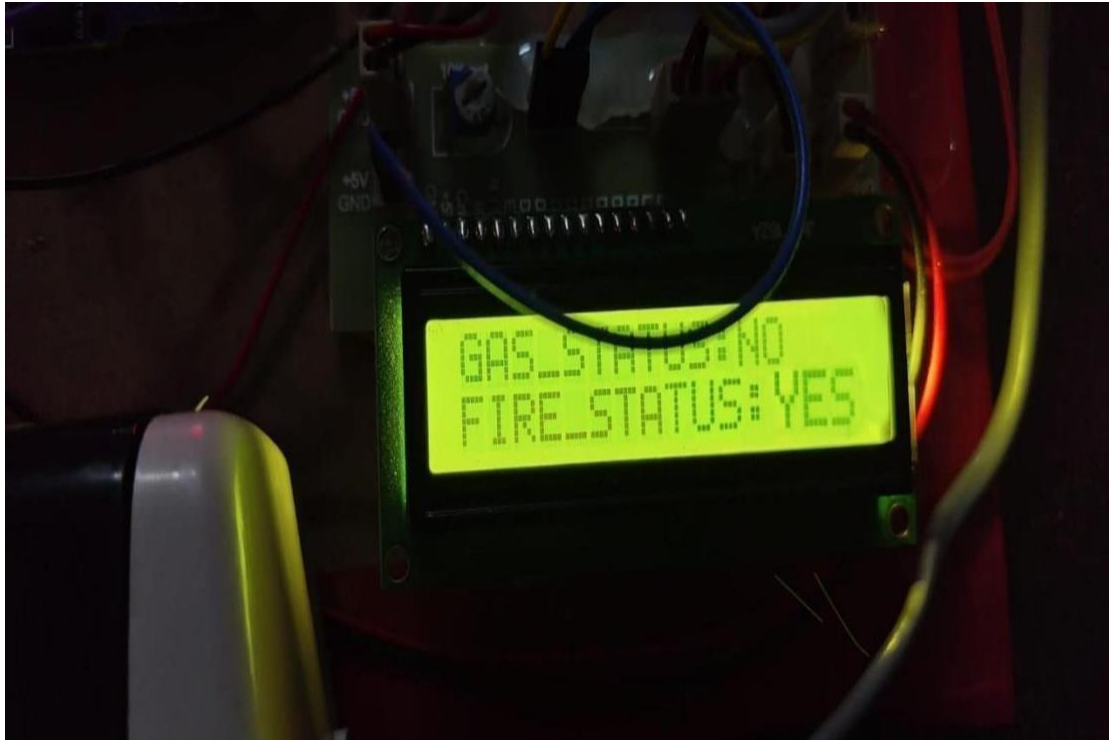




Screenshot 6.5 Exhaustible Fan is turned on when Gas leakage is detected



Screenshot 6.6 LCD display when no Gas leak and fire is not detected

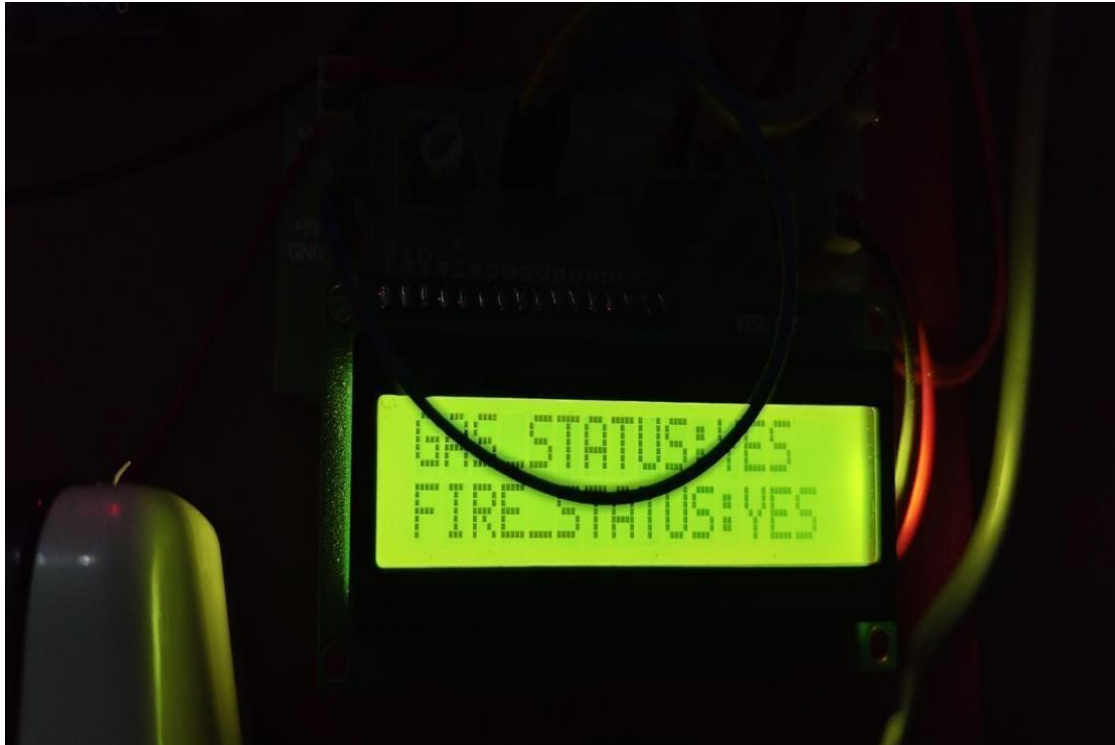


Screenshot 6.7 LCD display when Gas leak is not detected but fire is detected





Screenshot 6.8 LCD display when Gas leak is detected but fire is not detected



Screenshot 6.9 LCD display when Gas leak and fire is detected



Screenshot 6.10 Web Page for gas and fire status monitoring

## 7. CONCLUSION AND FUTURE SCOPE

## 7. Conclusion and Future Scope

### 7.1 Conclusion

The LPG Gas Leakage Detection System effectively detects gas leaks and activates safety mechanisms like buzzer, fan, and water sprinkler. It ensures enhanced household safety through automatic hazard control using Arduino.

### 7.2 Future Scope

- Developing an Android/iOS app that connects to the system via cloud (like Firebase or MQTT) so that users receive remote alerts.
- Adding a rechargeable battery backup so that the system can function even during power cuts.

## 8. REFERENCES

---

## 8. References

1. B. B. Kadaru, M. Usha, M. P. Pramod, M. Abhinay and M. S. S. Subramanyam, "IOT Based Gas Leakage Detection," 2024 International Conference on Emerging Systems and Intelligent Computing (ESIC), Bhubaneswar, India, 2024.
2. Subramanian, N. Selvam, S. Rajkumar, R. Mahalakshmi and J. Ramprabhakar, "Gas Leakage Detection System using IOT with integrated notifications using Pushbullet-A Review," 2020 Fourth International Conference on Inventive Systems and Control (ICISC), Coimbatore, 2020.
3. E. M. Saleh, T. Imbarake, S. Elakeili and A. G. Faraj, "SGD: Smart Gas Leakage Detection System for Home Safety," 2024 14th International Conference on Advanced Computer Information Technologies (ACIT), Ceske Budejovice, Czech Republic, 2024.